

PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN LANJUT
SEMESTER GENAP T.A. 2025/2026
LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NAMA/NIM : Azka Satria Nugroho / 123250214
Danang Nur Himawan / 123250222
KELAS/PLUG : IF-K
NAMA ASISTEN : Adhafa Joan Putranto / 123240069
Friska Rima Indriani / 123240111

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026

HALAMAN PENGESAHAN
LAPORAN PROYEK AKHIR

Disusun oleh :

Azka Satria Nugroho 123250214

Danang Nur Himawan 123250222

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Algoritma dan Pemrograman
Pada Tanggal : ...

Asisten Praktikum

Adhafa Joan Putranto

NIM. 123240069

Asisten Praktikum

Friska Rima Indriani

NIM. 123240111

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan Praktikum Pemrograman Web serta laporan akhir Pemrograman Web. Adapun laporan ini berisi tentang proyek akhir semester dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terima kasih kepada asisten dosen yang selalu membimbing dan mengajari kami dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun kami harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, kami ucapkan terima kasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 11 November 2023

Penyusun I

Penyusun II

Azka Satria Nugroho

NIM. 123250214

Danang Nur Himawan

NIM. 123250222

DAFTAR ISI

HALAMAN PENGESAHAN	i
KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I	1
1.1. Konsep Program	1
1.2. Fitur-Fitur Program	1
1.3. Topik-Topik Pemrograman yang Digunakan	2
BAB II.....	4
2.1. Tujuan Umum.....	4
2.2. Tujuan Khusus.....	4
BAB III	6
3.1. Implementasi Materi	6
3.1.1 Struct.....	6
3.1.2. Rekursi.....	6
3.1.3. Searching(Sequantial non Sentinel).....	6
3.1.4. Pointer.....	7
3.1.5. Sorting(Selection Sort)	7
3.1.6. File I/O (Simpan & Baca File)	8
3.2. Penjelasan Fitur.....	9
3.2.1. Fitur Login.....	9
3.2.2 . Fitur Menu Utama	9
3.2.3. Fitur Tambah Data.....	10
3.2.4. Fitur Tampilkan Data	11
3.2.5. Fitur Cari Data	12
3.2.6. Fitur Urutkan Data/Sorting.....	13
3.2.7. Fitur Simpan File	14
3.2.7. Fitur Baca File	15
3.3. Screnshoot Program	17
.....	17
3.4. Source Code Lengkap.....	21
BAB IV	31
4.1. Jadwal Pengerjaan.....	31

4.2. Pembagian Tugas	31
BAB V	32
5.1. Kesimpulan	32
5.2. Saran	32
DAFTAR PUSTAKA	34

BAB I

MANAJEMEN WARGA

1.1. Konsep Program

Program Manajemen Warga merupakan aplikasi berbasis bahasa pemrograman C++ yang dirancang untuk membantu proses pengelolaan dan pendataan warga secara sederhana. Program ini dibuat dengan memanfaatkan konsep-konsep yang telah dipelajari selama Praktikum Algoritma dan Pemrograman Lanjut, seperti struct, array, pointer, fungsi, rekursif, searching, sorting, serta file handling.

Dalam kehidupan sehari-hari, pendataan warga merupakan salah satu kegiatan administrasi yang penting. Data warga perlu disimpan dan dikelola dengan baik agar mudah dicari, diperbarui, maupun digunakan untuk berbagai keperluan. Oleh karena itu, program ini dibuat untuk mensimulasikan sistem pendataan warga yang dapat digunakan untuk menyimpan informasi penduduk secara terstruktur.

a. Data penduduk yang dikelola dalam program meliputi:

- 1) Nama Warga
- 2) Umur Warga
- 3) Alamat Warga
- 4) Jenis kelamin
- 5) Golongan darah

Program Program diawali dengan proses registrasi akun dan login pengguna. Setelah berhasil login, pengguna dapat mengakses menu utama yang berisi berbagai fitur pengelolaan data warga. Seluruh data disimpan sementara dalam array bertipe struct selama program berjalan dan dapat disimpan ke file agar tidak hilang ketika program ditutup. Melalui proyek ini, berbagai konsep dasar algoritma dan struktur data dapat diterapkan dalam sebuah aplikasi yang memiliki fungsi nyata dan mudah dipahami.

1.2. Fitur-Fitur Program

Program Manajemen Warga memiliki beberapa fitur utama sebagai berikut:

a. Registrasi Akun

Pengguna diminta membuat username dan password sebelum menggunakan program.

b. Login Sistem

Program melakukan validasi username dan password yang telah dibuat sebelumnya. Pengguna diberikan maksimal tiga kali kesempatan login.

c. Tambah Data Penduduk

Digunakan untuk menambahkan satu atau beberapa data warga ke dalam sistem.

d. Tampilkan Data Penduduk

Menampilkan seluruh data yang tersimpan dalam bentuk tabel yang rapi.

e. Cari Data Penduduk

Mencari data berdasarkan nama warga dengan metode pencarian string yang tidak membedakan huruf besar dan kecil.

f. Urutkan Data Penduduk

Mengurutkan data berdasarkan:

- 1) Umur terkecil ke terbesar
- 2) Umur terbesar ke terkecil
- 3) Nama A sampai Z

g. Hapus Data Penduduk

Menghapus data warga berdasarkan nama dan umur yang dimasukkan pengguna.

h. Simpan Data ke File

Menyimpan seluruh data warga ke file teks.

i. Baca Data dari File

Membaca data yang sebelumnya telah disimpan ke file.

j. Perhitungan Total Umur

Menghitung total umur seluruh warga menggunakan teknik rekursif.

1.3. Topik-Topik Pemrograman yang Digunakan

Beberapa materi praktikum yang diterapkan dalam proyek ini meliputi:

- a. Struct
- b. Array
- c. Pointer
- d. Function
- e. Rekursif
- f. Searching
- g. Sorting
- h. File Handling

- i. Percabangan
- j. Perulangan
- k. Validasi Input
- l. String Manipulation

Penerapan berbagai materi tersebut menjadikan program tidak hanya berfungsi sebagai aplikasi pendataan warga, tetapi juga sebagai sarana penerapan konsep-konsep pemrograman yang telah dipelajari selama satu semester.

BAB II

TUJUAN PROYEK AKHIR

2.1. Tujuan Umum

Proyek akhir ini dibuat sebagai sarana untuk mengimplementasikan materi-materi yang telah dipelajari selama Praktikum Algoritma dan Pemrograman Lanjut ke dalam sebuah aplikasi yang dapat digunakan untuk membantu pengelolaan data penduduk. Melalui proyek ini, mahasiswa diharapkan mampu memahami dan menerapkan konsep-konsep pemrograman dalam bentuk program yang terstruktur, fungsional, dan sesuai dengan kebutuhan pengguna.

Selain itu, proyek ini bertujuan untuk melatih kemampuan dalam menganalisis permasalahan, merancang solusi, serta mengembangkan aplikasi menggunakan bahasa pemrograman C++ dengan memanfaatkan berbagai materi yang telah dipelajari selama praktikum.

2.2. Tujuan Khusus

Tujuan khusus yang ingin dicapai dalam pembuatan program Manajemen Warga adalah sebagai berikut:

- a. Mengimplementasikan penggunaan struct untuk menyimpan data penduduk yang terdiri dari beberapa atribut.
- b. Mengimplementasikan array sebagai media penyimpanan data dalam jumlah banyak.
- c. Mengimplementasikan fungsi dan pointer untuk mempermudah pengelolaan program.
- d. Mengimplementasikan rekursif untuk melakukan perhitungan total umur seluruh penduduk.
- e. Mengimplementasikan algoritma searching dalam proses pencarian data warga berdasarkan nama.
- f. Mengimplementasikan algoritma selection sort dalam proses pengurutan data warga.
- g. Mengimplementasikan file handling untuk menyimpan dan membaca data dari file.
- h. Mengembangkan aplikasi berbasis menu yang mudah digunakan.

- i. Meningkatkan pemahaman mengenai penerapan algoritma dan struktur data dalam kasus nyata.
- j. Mengintegrasikan seluruh materi praktikum ke dalam satu proyek akhir yang utuh.

BAB III

PEMBAHASAN

3.1. Implementasi Materi

3.1.1 Struct

Struct digunakan untuk mendefinisikan tipe data Penduduk yang mengelompokkan beberapa atribut berbeda dalam satu kesatuan data terstruktur.

```
struct Penduduk
{
    string nama;
    int umur;
    string alamat;
    string jenis_kelamin;
    string gol_darah;
};
```

3.1.2. Rekursi

Rekursi diimplementasikan pada fungsi `totalUmur()` yang menghitung jumlah seluruh umur penduduk secara rekursif, yaitu fungsi memanggil dirinya sendiri dengan ukuran data yang berkurang hingga mencapai kondisi basis `n == 0`.

```
int totalUmur(Penduduk *data, int n)
{
    if (n == 0)
        return 0;
    return data[n - 1].umur + totalUmur(data, n - 1);
}
```

3.1.3. Searching(Sequantial non Sentinel)

Sequential Search Non Sentinel diimplementasikan pada fungsi `hapus_datawarga()`. Pencarian dilakukan dengan mengiterasi array dari index 0 hingga `n-1` menggunakan kondisi `i < n` sebagai pembatas loop. Jika data ditemukan, elemen setelahnya akan disalin secara berurutan ke posisi sebelumnya menggunakan `data[j] = data[j+1]`, kemudian jumlah data dikurangi dengan `n--`.

```
for (int i = 0; i < n; i++)
{
    if (data[i].nama == cari_nama && data[i].umur == cari_umur)
    {
        for (int j = i; j < n - 1; j++)
        {
            data[j] = data[j + 1];
        }
    }
}
```

```

        n--;
        ketemu = true;
        break;
    }
}

```

3.1.4. Pointer

Pointer diimplementasikan pada fungsi-fungsi yang menerima parameter `Penduduk *data`. Ketika fungsi dipanggil dari `main()`, program tidak mengirimkan seluruh isi data penduduk ke dalam fungsi, melainkan hanya mengirimkan **alamat memori** tempat array data tersimpan.

```

int totalUmur(Penduduk *data, int n)

void tampilkan(Penduduk *data, int n)

void tambahData(Penduduk *data, int &n)

void hapus_datawarga(Penduduk *data, int &n)

void cariData(Penduduk *data, int n)

void sorting(Penduduk *data, int n)

void simpanFile(Penduduk *data, int n)

void bacaFile(Penduduk *data, int &n)

```

3.1.5. Sorting(Selection Sort)

Selection Sort diimplementasikan pada fungsi `sorting()` dengan tiga pilihan pengurutan yaitu umur ascending, umur descending, dan nama A-Z. Algoritma ini bekerja dengan cara **mencari elemen terkecil atau terbesar** dari sisa array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini.

```

for (int i = 0; i < n - 1; i++)
{
    int idx = i;
    for (int j = i + 1; j < n; j++)
    {
        bool kondisi = false;
        if (pilihSort == 1)
            kondisi = data[j].umur < data[idx].umur; // ascending
        else if (pilihSort == 2)
            kondisi = data[j].umur > data[idx].umur; // descending
        else if (pilihSort == 3)
            kondisi = data[j].nama < data[idx].nama; // A-Z

        if (kondisi)

```

```

idx = j;
}
if (idx != i)
swap(data[i], data[idx]);
}

```

3.1.6. File I/O (Simpan & Baca File)

Fitur simpan dan baca file diimplementasikan menggunakan `ofstream` untuk menyimpan data dan `ifstream` untuk membaca data pada fungsi `simpanFile()` dan `bacaFile()`.

Simpan File:

```

ofstream file("data_penduduk.txt");
if (!file)
{
    cout << "Gagal membuka file!\n";
    return;
}

file << n << "\n";
for (int i = 0; i < n; i++)
{
    file << data[i].nama << "\n";
    file << data[i].umur << "\n";
    file << data[i].alamat << "\n";
    file << data[i].jenis_kelamin << "\n";
    file << data[i].gol_darah << "\n";
}
file.close();

```

Baca File:

```

ifstream file("data_penduduk.txt");
if (!file)
{
    cout << "File tidak ditemukan!\n";
    return;
}
n = 0;
file >> jumlah;
file.ignore();
for (int i = 0; i < jumlah && i < MAX; i++)
{
    getline(file, data[n].nama);
    file >> data[n].umur;
    file.ignore();
    getline(file, data[n].alamat);
    getline(file, data[n].jenis_kelamin);
    getline(file, data[n].gol_darah);
}

```

```

        n++;
    }
    file.close();

```

3.2. Penjelasan Fitur

3.2.1. Fitur Login

Fitur login diimplementasikan di `main()` sebagai gerbang masuk program. Pengguna terlebih dahulu membuat akun dengan memasukkan username dan password, kemudian melakukan login. Program memberikan kesempatan login sebanyak 3 kali, jika gagal semua maka program akan berhenti otomatis.

```

for (int i = 3; i > 0; i--)
{
    cout << "Masukkan Username Anda: ";
    getline(cin, username_login);
    cout << "Masukkan Pasword Anda: ";
    getline(cin, pasword_login);

    if (username_login != username_daftar && pasword_login !=
        username_pasword)
        cout << "Username dan Pasword Anda Tidak Valid!, Sisa
Kesempatan " << i - 1 << " Lagi\n";
    else
    {
        cout << "Login Berhasil!\n";
        berhasil = true;
        break;
    }
}
}

```

3.2.2. Fitur Menu Utama

Fitur menu utama diimplementasikan pada fungsi `tampilMenu()` yang dipanggil setiap kali program kembali ke halaman utama. Fungsi ini menampilkan seluruh pilihan fitur yang tersedia dalam program Sistem Informasi Penduduk.

```

void tampilMenu()
{
    cout <<
    "\n===== \n";
    cout << "          SISTEM INFORMASI PENDUDUK\n";
    cout << "===== \n";
    cout << "| 1. Tambah Data | \n";
    cout << "| 2. Tampilkan Data | \n";
    cout << "| 3. Cari Data | \n";
    cout << "| 4. Urutkan Data (Sorting) | \n";
    cout << "| 5. Hapus Data | \n";
    cout << "| 6. Simpan ke File | \n";
}

```

```

    cout << "| 7. Baca dari File                                |\n";
    cout << "| 8. Keluar                                              |\n";
    cout << "===== \n";
    cout << "Pilih menu: ";
}

```

3.2.3. Fitur Tambah Data

Fitur tambah data diimplementasikan pada fungsi `tambahData()`. Fitur ini memungkinkan pengguna menambahkan data penduduk baru ke dalam sistem. Pengguna dapat menambahkan lebih dari satu data sekaligus dengan menentukan jumlah data yang ingin dimasukkan terlebih dahulu.

```

void tambahData(Penduduk *data, int &n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                INPUT DATA PENDUDUK\n";
    cout << "===== \n";
    if (n >= MAX)
    {
        cout << "Data sudah penuh (maksimal " << MAX << ")!\n";
        return;
    }
    int jumlah;
    cout << "Jumlah data yang ingin ditambah: ";
    cin >> jumlah;
    cin.ignore();
    if (jumlah <= 0 || n + jumlah > MAX)
    {
        cout << "Jumlah tidak valid atau melebihi kapasitas!\n";
        return;
    }
    for (int i = 0; i < jumlah; i++)
    {
        cout << "\n--- Data ke-" << n + 1 << " ---\n";

        cout << "Nama          : ";
        getline(cin, data[n].nama);
        cout << "Umur           : ";
        while (!(cin >> data[n].umur) || data[n].umur < 0)
        {
            cout << "Input tidak valid. Masukkan umur (angka): ";
            cin.clear();
            cin.ignore(1000, '\n');
        }
        cin.ignore();
        cout << "Alamat         : ";
        getline(cin, data[n].alamat);
        cout << "Jenis Kelamin: ";
        getline(cin, data[n].jenis_kelamin);
    }
}

```

```

        cout << "Gol. Darah   : ";
        getline(cin, data[n].gol_darah);
        n++;
        cout << "Data berhasil ditambahkan!\n";
    }
}

    cout << "Data berhasil ditambahkan!\n";
}
}

```

3.2.4. Fitur Tampilkan Data

Fitur tampilkan data diimplementasikan pada fungsi `tampilkan()`. Fitur ini menampilkan seluruh data penduduk yang tersimpan dalam bentuk tabel terformat. Di bagian bawah tabel juga ditampilkan total jumlah data dan total umur seluruh penduduk.

```

void tampilkan(Penduduk *data, int n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                      TAMPIL DATA PENDUDUK \n";
    cout << "===== \n";

    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari. \n";
        return;
    }

    cout << "\n===== DATA PENDUDUK
===== \n";
    cout << left
        << setw(5) << "No"
        << setw(22) << "Nama"
        << setw(6) << "Umur"
        << setw(22) << "Alamat"
        << setw(16) << "Jenis Kelamin"
        << setw(10) << "Gol. Darah" << endl;
    cout << "-----
----- \n";

    for (int i = 0; i < n; i++)
    {
        cout << left
            << setw(5) << i + 1
            << setw(22) << data[i].nama
            << setw(6) << data[i].umur
            << setw(22) << data[i].alamat
            << setw(16) << data[i].jenis_kelamin
            << setw(10) << data[i].gol_darah << endl;
    }
}

```



```

        cout << "-----\n";
        cout << "Total data: " << n << " | Total umur: " <<
totalUmur(data, n) << "\n";
    }

```

3.2.5. Fitur Cari Data

Fitur cari data diimplementasikan pada fungsi `cariData()`. Fitur ini memungkinkan pengguna mencari data penduduk berdasarkan nama. Pencarian bersifat case-insensitive dan mendukung pencarian sebagian nama (substring), sehingga pengguna tidak perlu mengetikkan nama secara lengkap dan tepat.

```

void cariData(Penduduk *data, int n)
{
    system("cls");
    cout <<
"\n===== \n";
    cout << "                CARI DATA PENDUDUK\n";
    cout << "===== \n";

    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari.\n";
        return;
    }

    string cari, ulang;
    do
    {
        cout << "\nMasukkan nama yang dicari: ";
        getline(cin, cari);

        bool found = false;

        // Tampilkan header tabel
        cout << "\n\nHasil Pencarian : ";
        cout <<
"\n=====
===== \n";
        cout << left
            << setw(5) << "No"
            << setw(22) << "Nama"
            << setw(6) << "Umur"
            << setw(22) << "Alamat"
            << setw(16) << "Jenis Kelamin"
            << setw(10) << "Gol. Darah" << endl;
        cout << "-----
----- \n";

        int count = 0;
        for (int i = 0; i < n; i++)

```

```

{
    // Pencarian case-insensitive (cek substring)
    string namaLower = data[i].nama;
    string cariLower = cari;
    for (char &c : namaLower)
        c = tolower(c);
    for (char &c : cariLower)
        c = tolower(c);

    if (namaLower.find(cariLower) != string::npos)
    {
        found = true;
        count++;
        cout << left
            << setw(5) << count
            << setw(22) << data[i].nama
            << setw(6) << data[i].umur
            << setw(22) << data[i].alamat
            << setw(16) << data[i].jenis_kelamin
            << setw(10) << data[i].gol_darah << endl;
    }
}

if (!found)
{
    cout << "    Data dengan nama \"" << cari << "\" tidak
ditemukan.\n";
}
else
{
    cout << "-----\n";
    cout << "Ditemukan " << count << " data.\n";
}

cout << "\nCari lagi? (ya/tidak): ";
getline(cin, ulang);

} while (ulang == "ya" || ulang == "YA" || ulang == "Ya");
}

```

3.2.6. Fitur Urutkan Data/Sorting

Fitur urutkan data diimplementasikan pada fungsi `sorting()`. Fitur ini mengurutkan data penduduk menggunakan algoritma Selection Sort dengan tiga pilihan pengurutan yaitu berdasarkan umur ascending, umur descending, dan nama A-Z.

```

void sorting(Penduduk *data, int n)
{
    system("cls");
    cout <<
    "\n===== \n";

```

```

cout << "                                URUTKAN DATA PENDUDUK\n";
cout << "===== \n";
if (n == 0)
{
    cout << "[ERROR] Belum ada data untuk dicari.\n";
    return;
}

int pilihSort;
cout << "\nUrutkan berdasarkan:\n";
cout << "1. Umur (ascending)\n";
cout << "2. Umur (descending)\n";
cout << "3. Nama (A-Z)\n";
cout << "Pilih: ";
cin >> pilihSort;
cin.ignore();

// Selection sort
for (int i = 0; i < n - 1; i++)
{
    int idx = i;
    for (int j = i + 1; j < n; j++)
    {
        bool kondisi = false;
        if (pilihSort == 1)
            kondisi = data[j].umur < data[idx].umur;
        else if (pilihSort == 2)
            kondisi = data[j].umur > data[idx].umur;
        else if (pilihSort == 3)
            kondisi = data[j].nama < data[idx].nama;

        if (kondisi)
            idx = j;
    }
    if (idx != i)
        swap(data[i], data[idx]);
}

cout << "Data berhasil diurutkan!\n";
tampilkan(data, n);
}

```

3.2.7. Fitur Simpan File

Fitur simpan file diimplementasikan pada fungsi `simpanFile()`. Fitur ini memungkinkan pengguna menyimpan seluruh data penduduk yang ada di dalam sistem ke dalam file `data_penduduk.txt` secara permanen sehingga data tidak hilang saat program ditutup.

```

void simpanFile(Penduduk *data, int n)
{
    system("cls");
}

```

```

        cout <<
"\n=====\\n";
        cout << "                SIMPAN FILE DATA PENDUDUK\\n";
        cout << "=====\\n";
        if (n == 0)
        {
            cout << "[ERROR] Data Belum Ada Yang Di Masukkan.\\n";
            return;
        }

        ofstream file("data_penduduk.txt");
        if (!file)
        {
            cout << "Gagal membuka file untuk menyimpan!\\n";
            return;
        }

        // Simpan jumlah data di baris pertama supaya bacaFile tahu berapa
        record
        file << n << "\\n";

        for (int i = 0; i < n; i++)
        {
            file << data[i].nama << "\\n";
            file << data[i].umur << "\\n";
            file << data[i].alamat << "\\n";
            file << data[i].jenis_kelamin << "\\n";
            file << data[i].gol_darah << "\\n";
        }

        file.close();
        cout << "Data berhasil disimpan ke \"data_penduduk.txt\" (" << n
<< " record).\\n";
    }
}

```

3.2.7. Fitur Baca File

Fitur baca file diimplementasikan pada fungsi `bacaFile()`. Fitur ini memungkinkan pengguna memuat kembali data penduduk yang sebelumnya telah disimpan di file `data_penduduk.txt` ke dalam sistem.

```

void bacaFile(Penduduk *data, int &n)
{
    system("cls");
    cout <<
"\n=====\\n";
    cout << "                BACA FILE DATA PENDUDUK\\n";
    cout << "=====\\n";
    ifstream file("data_penduduk.txt");

    if (!file)
    {
        cout << "File \"data_penduduk.txt\" tidak ditemukan!\\n";
    }
}

```

```
        return;
    }

    // Reset data lama
    n = 0;

    // Baca jumlah record dari baris pertama
    int jumlah;
    file >> jumlah;
    file.ignore();

    if (jumlah <= 0 || jumlah > MAX)
    {
        cout << "Isi file tidak valid atau kosong.\n";
        return;
    }

    for (int i = 0; i < jumlah && i < MAX; i++)
    {
        if (!getline(file, data[n].nama))
            break;
        file >> data[n].umur;
        file.ignore();
        if (!getline(file, data[n].alamat))
            break;
        if (!getline(file, data[n].jenis_kelamin))
            break;
        if (!getline(file, data[n].gol_darah))
            break;
        n++;
    }

    file.close();
    cout << "Berhasil membaca " << n << " data dari file.\n";
}
```

3.3. Screenshoot Program

```
=====
                          PEMBUATAN AKUN
=====

Masukkan Username: Xiaom
Masukkan Pasword: 123

=====
                          LOGIN AKUN
=====

Masukkan Username Anda: Xiaom
Masukkan Pasword Anda: 123

Login Berhasil, Selamat Datang Xiaom
```

Gambar 1 Login dan Pembuatan Akun

```
=====
Selamat datang di Sistem Informasi Penduduk
=====

=====
SISTEM INFORMASI PENDUDUK
=====

| 1. Tambah Data          |
| 2. Tampilkan Data       |
| 3. Cari Data            |
| 4. Urutkan Data (Sorting)|
| 5. Hapus Data           |
| 6. Simpan ke File       |
| 7. Baca dari File       |
| 8. Keluar               |
|                          |
=====
Pilih menu: █
```

Gambar 2 Tampilkan Menu

```

=====
                        INPUT DATA PENDUDUK
=====
Jumlah data yang ingin ditambah: 3

--- Data ke-1 ---
Nama      : Danang Nur Himawan
Umur      : 19
Alamat    : Purworejo
Jenis Kelamin: laki-laki
Gol. Darah : 0
Data berhasil ditambahkan!

--- Data ke-2 ---
Nama      : Azka Satria Nugroho
Umur      : 18
Alamat    : Kelaten
Jenis Kelamin: laki-laki
Gol. Darah : 0
Data berhasil ditambahkan!

--- Data ke-3 ---
Nama      : Dana Wikara
Umur      : 20
Alamat    : Bantul
Jenis Kelamin: laki-laki
Gol. Darah : A
Data berhasil ditambahkan!

[Tekan Enter untuk kembali ke menu...]

```

Gambar 3 Input Data Warga

```

=====
                        TAMPIL DATA PENDUDUK
=====

===== DATA PENDUDUK =====
No   Nama                Umur  Alamat        Jenis Kelamin  Gol. Darah
-----
1    Danang Nur Himawan   19    Purworejo     laki-laki      0
2    Azka Satria Nugroho  18    Kelaten       laki-laki      0
3    Dana Wikara          20    Bantul        laki-laki      A
-----
Total data: 3 | Total umur: 57

[Tekan Enter untuk kembali ke menu...]

```

Gambar 4 Tampilkan Data Warga

```

=====
CARI DATA PENDUDUK
=====

Masukkan nama yang dicari: Danang Nur Himawan

Hasil Pencarian :
=====
No      Nama                Umur  Alamat                Jenis Kelamin  Gol. Darah
-----
1      Danang Nur Himawan    19    Purworejo             laki-laki      0
-----
Ditemukan 1 data.

Cari lagi? (ya/tidak): 

```

Gambar 5 Cari Data Warga

```

=====
URUTKAN DATA PENDUDUK
=====

Urutkan berdasarkan:
1. Umur (ascending)
2. Umur (descending)
3. Nama (A-Z)
Pilih: 1

```

Gambar 6 dan 7 Sorting Data Warga

```

=====
TAMPIL DATA PENDUDUK
=====

===== DATA PENDUDUK =====
No      Nama                Umur  Alamat                Jenis Kelamin  Gol. Darah
-----
1      Azka Satria Nugroho  18    Kelaten              laki-laki      0
2      Danang Nur Himawan   19    Purworejo            laki-laki      0
3      Dana Wikara          20    Bantul               laki-laki      A
-----
Total data: 3 | Total umur: 57

[Tekan Enter untuk kembali ke menu...]

```

Gambar 6 dan 7 Sorting Data Warga

```

=====
HAPUS DATA PENDUDUK
=====

Masukkan Nama Warga: Danang Nur Himawan
Masukkan Umur Warga: 19

```

Gambar 8 dan 9 Hapus Data Warga


```

=====
TAMPIL DATA PENDUDUK
=====

===== DATA PENDUDUK =====
No    Nama                Umur  Alamat                Jenis Kelamin  Gol. Darah
-----
1     Azka Satria Nugroho    18    Kelaten                laki-laki      0
2     Dana Wikara            20    Bantul                 laki-laki      A

Total data: 2 | Total umur: 38

[Tekan Enter untuk kembali ke menu...]

```

Gambar 8 dan 9 Hapus Data Warga

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=====
SIMPAN FILE DATA PENDUDUK
=====
Data berhasil disimpan ke "data_penduduk.txt" (2 record).

[Tekan Enter untuk kembali ke menu...]

```

Gambar 10 dan 11 Simpan File

```

data_penduduk.txt
1      2
2      Azka Satria Nugroho
3      18
4      Kelaten
5      laki-laki
6      0
7      Dana Wikara
8      20
9      Bantul
10     laki-laki
11     A
12

```

Gambar 10 dan 11 Simpan File

```

=====
TAMPIL DATA PENDUDUK
=====

===== DATA PENDUDUK =====
No    Nama                Umur  Alamat                Jenis Kelamin  Gol. Darah
-----
1     Azka Satria Nugroho    18    Kelaten                laki-laki      0
2     Dana Wikara            20    Bantul                 laki-laki      A

Total data: 2 | Total umur: 38

[Tekan Enter untuk kembali ke menu...]

```

Gambar 12 Baca File

3.4. Source Code Lengkap

```

#include <iostream>
#include <iomanip>
#include <fstream>
#include <string>
using namespace std;

// ===== STRUCT =====
struct Penduduk
{
    string nama;
    int umur;
    string alamat;
    string jenis_kelamin;
    string gol_darah;
};

const int MAX = 100;

// ===== REKURSIF =====
int totalUmur(Penduduk *data, int n)
{
    if (n == 0)
        return 0;
    return data[n - 1].umur + totalUmur(data, n - 1);
}

// ===== TAMPIL =====
void tampilkan(Penduduk *data, int n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                      TAMPIL DATA PENDUDUK \n";
    cout << "===== \n";

    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari. \n";
        return;
    }

    cout << "\n===== DATA PENDUDUK
===== \n";
    cout << left
        << setw(5) << "No"
        << setw(22) << "Nama"
        << setw(6) << "Umur"
        << setw(22) << "Alamat"
        << setw(16) << "Jenis Kelamin"
        << setw(10) << "Gol. Darah" << endl;
    cout << "-----
----- \n";

```

```

for (int i = 0; i < n; i++)
{
    cout << left
        << setw(5) << i + 1
        << setw(22) << data[i].nama
        << setw(6) << data[i].umur
        << setw(22) << data[i].alamat
        << setw(16) << data[i].jenis_kelamin
        << setw(10) << data[i].gol_darah << endl;
}
cout << "-----\n";
cout << "Total data: " << n << " | Total umur: " <<
totalUmur(data, n) << "\n";
}

// ===== TAMBAH =====
void tambahData(Penduduk *data, int &n)
{
    system("cls");
    cout <<
"\n===== \n";
    cout << "                INPUT DATA PENDUDUK\n";
    cout << "===== \n";
    if (n >= MAX)
    {
        cout << "Data sudah penuh (maksimal " << MAX << ")!\n";
        return;
    }

    int jumlah;
    cout << "Jumlah data yang ingin ditambah: ";
    cin >> jumlah;
    cin.ignore();

    if (jumlah <= 0 || n + jumlah > MAX)
    {
        cout << "Jumlah tidak valid atau melebihi kapasitas!\n";
        return;
    }

    for (int i = 0; i < jumlah; i++)
    {
        cout << "\n--- Data ke-" << n + 1 << " ---\n";

        cout << "Nama          : ";
        getline(cin, data[n].nama);

        cout << "Umur          : ";
        while (!(cin >> data[n].umur) || data[n].umur < 0)
        {
            cout << "Input tidak valid. Masukkan umur (angka): ";
            cin.clear();

```

```

        cin.ignore(1000, '\n');
    }
    cin.ignore();

    cout << "Alamat      : ";
    getline(cin, data[n].alamat);

    cout << "Jenis Kelamin: ";
    getline(cin, data[n].jenis_kelamin);

    cout << "Gol. Darah   : ";
    getline(cin, data[n].gol_darah);

    n++;
    cout << "Data berhasil ditambahkan!\n";
}
}

// ===== HAPUS =====
void hapus_datawarga(Penduduk *data, int &n)
{
    system("cls");

    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari.\n";
        return;
    }

    cout <<
    "\n===== \n";
    cout << "                HAPUS DATA PENDUDUK\n";
    cout << "===== \n";

    string cari_nama;
    int cari_umur;

    cout << "Masukkan Nama Warga: ";
    getline(cin, cari_nama);

    cout << "Masukkan Umur Warga: ";
    cin >> cari_umur;
    cin.ignore();

    bool ketemu = false;

    for (int i = 0; i < n; i++)
    {
        if (data[i].nama == cari_nama && data[i].umur == cari_umur)
        {
            for (int j = i; j < n - 1; j++)
            {
                data[j] = data[j + 1];
            }

```

```

        n--;
        cout << "[SUKSES] Data Berhasil Di Hapus\n";
        ketemu = true;
        tampilkan(data, n);
        break;
    }
}
if (!ketemu)
{
    cout << "[ERROR] Data Tidak Di Temukan\n";
}
}

// ===== SEARCH =====
void cariData(Penduduk *data, int n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                CARI DATA PENDUDUK\n";
    cout << "===== \n";

    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari.\n";
        return;
    }

    string cari, ulang;
    do
    {
        cout << "\nMasukkan nama yang dicari: ";
        getline(cin, cari);

        bool found = false;

        // Tampilkan header tabel
        cout << "\n\nHasil Pencarian : ";
        cout <<
        "\n=====
===== \n";
        cout << left
            << setw(5) << "No"
            << setw(22) << "Nama"
            << setw(6) << "Umur"
            << setw(22) << "Alamat"
            << setw(16) << "Jenis Kelamin"
            << setw(10) << "Gol. Darah" << endl;
        cout << "-----
----- \n";

        int count = 0;
        for (int i = 0; i < n; i++)
        {

```

```

// Pencarian case-insensitive (cek substring)
string namaLower = data[i].nama;
string cariLower = cari;
for (char &c : namaLower)
    c = tolower(c);
for (char &c : cariLower)
    c = tolower(c);

if (namaLower.find(cariLower) != string::npos)
{
    found = true;
    count++;
    cout << left
        << setw(5) << count
        << setw(22) << data[i].nama
        << setw(6) << data[i].umur
        << setw(22) << data[i].alamat
        << setw(16) << data[i].jenis_kelamin
        << setw(10) << data[i].gol_darah << endl;
}
}

if (!found)
{
    cout << "    Data dengan nama \"" << cari << "\" tidak
ditemukan.\n";
}
else
{
    cout << "-----\n";
    cout << "Ditemukan " << count << " data.\n";
}

    cout << "\nCari lagi? (ya/tidak): ";
    getline(cin, ulang);

} while (ulang == "ya" || ulang == "YA" || ulang == "Ya");
}

// ===== SORT =====
void sorting(Penduduk *data, int n)
{
    system("cls");
    cout <<
"\n===== \n";
    cout << "                URUTKAN DATA PENDUDUK\n";
    cout << "===== \n";
    if (n == 0)
    {
        cout << "[ERROR] Belum ada data untuk dicari.\n";
        return;
    }
}

```

```

int pilihSort;
cout << "\nUrutkan berdasarkan:\n";
cout << "1. Umur (ascending)\n";
cout << "2. Umur (descending)\n";
cout << "3. Nama (A-Z)\n";
cout << "Pilih: ";
cin >> pilihSort;
cin.ignore();

// Selection sort
for (int i = 0; i < n - 1; i++)
{
    int idx = i;
    for (int j = i + 1; j < n; j++)
    {
        bool kondisi = false;
        if (pilihSort == 1)
            kondisi = data[j].umur < data[idx].umur;
        else if (pilihSort == 2)
            kondisi = data[j].umur > data[idx].umur;
        else if (pilihSort == 3)
            kondisi = data[j].nama < data[idx].nama;

        if (kondisi)
            idx = j;
    }
    if (idx != i)
        swap(data[i], data[idx]);
}

cout << "Data berhasil diurutkan!\n";
tampilkan(data, n);
}

// ===== FILE =====
void simpanFile(Penduduk *data, int n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                SIMPAN FILE DATA PENDUDUK\n";
    cout << "===== \n";
    if (n == 0)
    {
        cout << "[ERROR] Data Belum Ada Yang Di Masukkan.\n";
        return;
    }

    ofstream file("data_penduduk.txt");
    if (!file)
    {
        cout << "Gagal membuka file untuk menyimpan!\n";
        return;
    }
}

```

```

    // Simpan jumlah data di baris pertama supaya bacaFile tahu berapa
    record
    file << n << "\n";

    for (int i = 0; i < n; i++)
    {
        file << data[i].nama << "\n";
        file << data[i].umur << "\n";
        file << data[i].alamat << "\n";
        file << data[i].jenis_kelamin << "\n";
        file << data[i].gol_darah << "\n";
    }

    file.close();
    cout << "Data berhasil disimpan ke \"data_penduduk.txt\" (" << n
    << " record).\n";
}

void bacaFile(Penduduk *data, int &n)
{
    system("cls");
    cout <<
    "\n===== \n";
    cout << "                BACA FILE DATA PENDUDUK\n";
    cout << "===== \n";
    ifstream file("data_penduduk.txt");

    if (!file)
    {
        cout << "File \"data_penduduk.txt\" tidak ditemukan!\n";
        return;
    }

    // Reset data lama
    n = 0;

    // Baca jumlah record dari baris pertama
    int jumlah;
    file >> jumlah;
    file.ignore();

    if (jumlah <= 0 || jumlah > MAX)
    {
        cout << "Isi file tidak valid atau kosong.\n";
        return;
    }

    for (int i = 0; i < jumlah && i < MAX; i++)
    {
        if (!getline(file, data[n].nama))
            break;
        file >> data[n].umur;
        file.ignore();
    }
}

```



```

        if (!getline(file, data[n].alamat))
            break;
        if (!getline(file, data[n].jenis_kelamin))
            break;
        if (!getline(file, data[n].gol_darah))
            break;
        n++;
    }

    file.close();
    cout << "Berhasil membaca " << n << " data dari file.\n";
}

// ===== MENU =====
void tampilMenu()
{
    cout <<
    "\n===== \n";
    cout << "                SISTEM INFORMASI PENDUDUK\n";
    cout << "===== \n";
    cout << "| 1. Tambah Data | \n";
    cout << "| 2. Tampilkan Data | \n";
    cout << "| 3. Cari Data | \n";
    cout << "| 4. Urutkan Data (Sorting) | \n";
    cout << "| 5. Hapus Data | \n";
    cout << "| 6. Simpan ke File | \n";
    cout << "| 7. Baca dari File | \n";
    cout << "| 8. Keluar | \n";
    cout << "===== \n";
    cout << "Pilih menu: ";
}

// ===== MAIN =====
int main()
{
    system("cls");
    Penduduk data[MAX];
    string username_daftar, username_pasword;
    int n = 0;
    int pilihan;

    cout <<
    "\n===== \n";
    cout << "                PEMBUATAN AKUN
\n";
    cout <<
    "===== \n";

    cout << "\nMasukkan Username: ";
    getline(cin, username_daftar);

    cout << "Masukkan Pasword: ";
    getline(cin, username_pasword);

```

```

bool berhasil = false;

for (int i = 3; i > 0; i--)
{
    string username_login, password_login;

    cout <<
"\n=====\\n";
    cout << "                LOGIN AKUN
\\n";
    cout <<
"=====\\n";
    cout << "\\nMasukkan Username Anda: ";
    getline(cin, username_login);

    cout << "Masukkan Pasword Anda: ";
    getline(cin, password_login);

    if (username_login != username_daftar && password_login !=
username_pasword)
        cout << "Username dan Pasword Anda Tidak Valid!, Sisa
Kesempatan " << i - 1 << " Lagi" << endl;
    else if (username_login != username_daftar)
        cout << "username Anda Tidak Sesuai!, Sisa Kesempatan " <<
i - 1 << " Lagi" << endl;
    else if (password_login != username_pasword)
        cout << "Pasword Anda Tidak Sesuai!, Sisa Kesempatan " <<
i - 1 << " Lagi" << endl;
    else
    {
        cout << "\\nLogin Berhasil, Selamat Datang " <<
username_login << endl;
        berhasil = true;
        break;
    }
}
if (!berhasil)
{
    cout << "\\nKesempatan Telah Habis!!, Program Selesai.\\n";
    return 0;
}

do
{
    cout << endl;
    cout <<
"\n=====\\n";
    cout << "        Selamat datang di Sistem Informasi Penduduk\\n";
    cout <<
"=====\\n";

    tampilMenu();

    while (!(cin >> pilihan))

```

```

{
    cout << "Input tidak valid. Masukkan angka menu: ";
    cin.clear();
    cin.ignore(1000, '\n');
}
cin.ignore();

cout << "\n";

switch (pilihan)
{
case 1:
    tambahData(data, n);
    break;
case 2:
    tampilkan(data, n);
    break;
case 3:
    cariData(data, n);
    break;
case 4:
    sorting(data, n);
    break;
case 5:
    hapus_datawarga(data, n);
    break;
case 6:
    simpanFile(data, n);
    break;
case 7:
    bacaFile(data, n);
    tampilkan(data, n);
    break;
case 8:
    cout << "Terima kasih telah menggunakan program ini!\n";
    break;
default:
    cout << "Pilihan tidak valid! Masukkan angka 1-7.\n";
}

if (pilihan != 8)
{
    cout << "\n[Tekan Enter untuk kembali ke menu...]";
    cin.get();
}

} while (pilihan != 8);

return 0;
}

```

BAB IV

JADWAL Pengerjaan TUGAS DAN PEMBAGIAN TUGAS

4.1. Jadwal Pengerjaan

Tabel 4.1 Tabel Jadwal Pengerjaan

No.	Kegiatan	MEI			
		1	2	3	4
1.	Mengkaji Masalah	✓			
2.	Perancangan Sistem Program		✓		
3.	Pembuatan Fitur Program		✓		
4.	Pengujian Program			✓	
5.	Penyusunan Laporan				✓

4.2. Pembagian Tugas

Kami membagi tugas sesuai dengan tanggung jawab masing masing setiap kegiatan.

Tabel 4.2 Tabel Pembagian Tugas

No.	Kegiatan	Penanggung Jawab
1.	Mengkaji Masalah	Azka Satria Nugroho
2.	Perancangan Sistem Program	Danang Nur Himawan
3.	Pembuatan Fitur Program	Azka dan Danang
4.	Pengujian Program	Azka dan Danang
5.	Penyusunan Laporan	Azka dan Danang

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, Program Manajemen Warga berhasil dikembangkan menggunakan bahasa pemrograman C++. PrSogram ini mampu membantu proses pengelolaan data penduduk secara sederhana melalui berbagai fitur yang tersedia, seperti penambahan data, penampilan data, pencarian data, pengurutan data, penghapusan data, penyimpanan data ke file, serta pembacaan data dari file.

Dalam proses pengembangan program, berbagai materi yang telah dipelajari selama Praktikum Algoritma dan Pemrograman Lanjut berhasil diterapkan ke dalam satu aplikasi yang terintegrasi. Penggunaan struct memudahkan penyimpanan data penduduk yang memiliki beberapa atribut, sedangkan array digunakan sebagai media penyimpanan data selama program berjalan. Implementasi pointer dan fungsi membuat program menjadi lebih terstruktur dan mudah dipelihara.

Program juga menerapkan fungsi rekursif untuk menghitung total umur seluruh penduduk yang tersimpan. Selain itu, algoritma searching digunakan untuk mempermudah pencarian data berdasarkan nama warga, sedangkan algoritma selection sort digunakan untuk mengurutkan data berdasarkan umur maupun nama. Penggunaan file handling memungkinkan data disimpan secara permanen sehingga dapat digunakan kembali pada saat program dijalankan di waktu yang berbeda.

Berdasarkan hasil pengujian, seluruh fitur utama program dapat berjalan sesuai dengan tujuan yang telah direncanakan. Program mampu mengelola data penduduk dengan baik dan memberikan pengalaman penggunaan yang cukup mudah melalui sistem menu interaktif yang tersedia. Dengan demikian, tujuan pembuatan proyek akhir ini dapat dikatakan telah tercapai dengan baik, baik dari sisi fungsionalitas program maupun penerapan konsep-konsep pemrograman yang dipelajari selama praktikum.

5.2. Saran

Meskipun program telah berjalan sesuai dengan kebutuhan dasar pengelolaan data penduduk, masih terdapat beberapa pengembangan yang dapat dilakukan pada masa mendatang agar program menjadi lebih baik dan lebih lengkap.

Beberapa saran pengembangan yang dapat dilakukan antara lain:

- a. Menambahkan fitur edit data sehingga pengguna dapat memperbarui data warga tanpa harus menghapus dan memasukkan data kembali.
- b. Menambahkan sistem penyimpanan berbasis database seperti MySQL atau SQLite agar pengelolaan data menjadi lebih aman dan mampu menampung data dalam jumlah yang lebih besar.
- c. Mengembangkan antarmuka program menjadi berbasis Graphical User Interface (GUI) sehingga lebih mudah digunakan dan lebih menarik dibandingkan tampilan berbasis console.
- d. Menambahkan fitur pencarian berdasarkan kriteria lain seperti alamat, golongan darah, atau jenis kelamin untuk meningkatkan fleksibilitas penggunaan program.
- e. Menambahkan sistem autentikasi yang lebih aman, seperti penyembunyian password saat login dan penyimpanan akun pada file atau database.
- f. Menambahkan fitur laporan statistik, misalnya jumlah warga berdasarkan jenis kelamin, rata-rata umur penduduk, atau jumlah warga berdasarkan golongan darah.
- g. Menambahkan validasi data yang lebih lengkap untuk mengurangi kemungkinan kesalahan input dari pengguna.
- h. Mengoptimalkan algoritma pencarian dan pengurutan apabila program digunakan untuk mengelola data dalam jumlah yang lebih besar.
- i. Menambahkan fitur backup dan restore data agar keamanan data lebih terjamin.
- j. Mengembangkan program menjadi aplikasi berbasis web atau mobile sehingga dapat diakses dengan lebih mudah oleh banyak pengguna.

Dengan adanya berbagai pengembangan tersebut, Program Manajemen Warga diharapkan dapat menjadi sistem pengelolaan data yang lebih lengkap, lebih efisien, dan lebih bermanfaat untuk digunakan dalam berbagai kebutuhan administrasi penduduk di masa mendatang.

DAFTAR PUSTAKA

<https://github.com/dhimawan617-oss/Projek-Alpro-Kelompok-2>

<https://github.com/azkov07/PROJECT--PRAK-ALPRO-LANJUT-KEL2>

KETENTUAN :

1. Huruf Times New Roman 12 (Hitam)
2. Keterangan tabel dan gambar Times New Roman 11 (Hitam)
3. Isi tabel Times New Roman 10 (Hitam)
4. Isi source code Source code pro 10 (Hitam)
5. Margin Top Left 3, Right Bottom 2,5
6. Spacing 1,5 dan remove space before and after paragraph
7. Penomoran halaman pada setiap awal bab di bawah tengah, setiap isi bab di atas kanan, dan halaman cover tanpa halaman.
8. Tulisan dalam format laporan ini yang diberi warna merah hanya berupa tanda dan dapat disesuaikan.